



Al-Imam Mohammad Ibn Saud Islamic University

College of Computer and Information Sciences

Computer Science Department

Riyadh Metro System

Course: CS 1251 Software Engineering 2
Course doctor: Ibrahim Abdel Moneim Abdel Majeed
Division number: 173
Group name: Saudi Metro Engineers

Rashid Al-Qahtani	444001334
Abdullah Alkhawlani	444000843
Khalid Aldosari	443018297
Khalid Altamimi	443014196
Mohammed Al-Fayez	444001229

Phase 1

Software Engineering 2

Risk Identification

In this section, we identify the potential risks that may affect the success and continuity of the Riyadh Train Integrated System project. These risks are categorized into three main types: technical risks, operational risks, and external risks. The identification was done through brainstorming sessions among the team and reviewing challenges from similar projects.

Technical Risks:

1. **System Integration Issues:** The system involves multiple modules (ticketing, scheduling, fare collection, etc.). These modules might not integrate smoothly, leading to communication errors or data mismatches.
2. **Software Bugs or Failures:** Since the system is complex and will operate in real-time, unexpected bugs or logic failures could disrupt essential functions like ticket validation or route planning.
3. **Limited Testing Time:** There might not be enough time to conduct thorough testing, which increases the risk of undiscovered bugs before deployment.

Operational Risks:

1. **User Misuse or Input Errors:** Passengers or metro staff might misuse the system or enter incorrect data, which could cause system crashes or confusion in scheduling.
2. **Unclear Requirements from Stakeholders:** In some cases, the project may lack clear, finalized requirements from authorities or clients, leading to repeated changes and delays.
3. **Inconsistent Internet Access in Stations:** If internet access is unstable in some stations, it might affect real-time functionalities like online ticket validation or route updates.

External Risks:

1. **Regulatory or Compliance Changes:** If government policies or transport regulations change during development, we may need to modify significant parts of the system.
2. **Supplier or Vendor Delays:** Any delay from hardware vendors (like card readers or scanners) could delay system integration and testing.
3. **Public Resistance to Change:** Users used to traditional systems might resist switching to a fully digital system, affecting adoption and usability.

Risk Analysis:

Impact and likelihood of each Risk:

Risk Impact and Likelihood Scale			
Risk	Impact	Likelihood	Outcome
System Integration Issues	Critical	Likely	Can cause the system to fully stop operating
Unclear Requirements from Stakeholders	Major	Most Likely	delays, rework, and confusion destable the project's workflow
Limited Testing Time	Major	Likely	might cause high-impact bugs go undetected
Supplier or Vendor Delays	Major	Likely	Delayed supplies deliveries slows down development
Regulatory or Compliance Changes	Major	Possible	might require design changes but it's not frequent
Software Bugs or Failures	Moderate	Likely	most aren't severe but some can cause major issues
User Misuse or Input Errors	Moderate	Likely	Can lead to data inconsistencies or confusion.
Public Resistance to Change	Moderate	Likely	makes adoption and satisfaction, but not system function.
Inconsistent Internet Access in Stations	Moderate	Possible	Affects real-time features

Risk Mitigation:

Mitigation Strategies for Technical Risks

1. System Integration Issues

Mitigation Strategy:

- **Standardized Interfaces:** Use well-defined APIs and protocols to ensure seamless communication between modules.
- **Incremental Integration:** Integrate modules one at a time, testing each integration thoroughly before adding the next.
- **Integration Testing:** Conduct regular integration tests to identify and resolve issues early.

Action Plan:

- Define API specifications for all modules by the end of the design phase.
- Schedule bi-weekly integration tests starting after the first module is developed.
- Assign a dedicated integration team to monitor and troubleshoot communication issues.

2. Software Bugs or Failures

Mitigation Strategy:

- **Robust Testing Framework:** Implement unit, functional, and stress testing to catch bugs before deployment.
- **Code Reviews:** Conduct peer reviews for all code to ensure quality and catch logical errors early.
- **Bug Tracking System:** Use a centralized system to log, prioritize, and resolve bugs efficiently.

Action Plan:

- Set up a testing suite with automated unit tests within the first month of development.

- Require at least one peer review per code submission, enforced via version control tools
- Adopt a bug tracking tool like Jira and review bug reports weekly.

3. Limited Testing Time

Mitigation Strategy:

- **Prioritized Testing:** Focus testing on critical components (e.g., ticketing, route planning) to ensure core functionality is robust.
- **Automated Testing:** Use automated testing tools to speed up repetitive test cases, freeing time for manual exploratory testing.
- **Early Testing:** Begin testing during development (shift-left testing) to catch issues sooner.

Action Plan:

- Create a testing priority list by ranking modules based on impact, finalized before coding starts.
- Implement automated testing tools like Selenium for UI and Postman for APIs within two months.
- Allocate 20% of development sprints for testing, starting from the first sprint.

Mitigation Strategies for Operational Risks

1. Input Validation & Error Handling

Mitigation Strategy:

- **Validation:** Implement strong validation rules to prevent incorrect data entry.
- **Pre-Defined Text:** Use auto-complete or drop-down menus instead of free text where possible.
- **False Input Protocol:** Smartly handle unexpected inputs to avoid system crashes.
-

Action Plan:

- Implement backend and frontend validation.
- Limit access to sensitive functions based on user roles.
- Prevent unauthorized users from performing high-impact actions

2. Establish a Clear Communication Plan:

Mitigation Strategy:

- **Decision making:** Define who the decision-makers are and how often you'll meet them.
- **Visuals:** Use visual tools like mockups, flowcharts, or prototypes to make abstract ideas more concrete.
- **Documentation and Approval:** Document requirements in detail and get sign-off before development starts.

Action Plan:

- Use Agile or Iterative Development.
- Break the project into smaller, manageable sprints so you can adjust to changes without derailing the entire project.
- Regularly use demos or progress reviews to help catch misunderstandings early.

3. Data Sync Mechanism

Mitigation Strategy:

- **Implement automatic sync:** once the device reconnects to the internet, it should automatically upload and sort out data.
- **Conflict Resolution:** Use conflict resolution rules to handle discrepancies when syncing.
- **Edge Servers:** Deploy at major stations to process data locally instead of relying on the cloud for every action.

Action Plan:

- Create a system that syncs data (like ticket scans) once the device reconnects to the internet.
- Combine Wi-Fi, cellular (4G/5G), and even mesh networks to improve reliability in areas with poor coverage.
- Add network backup options so it gives the system a second chance to stay connected when the main network fails.

Mitigation Strategies for External Risks

1. Regulatory or Compliance Changes

Mitigation Strategy:

- **Proactive Engagement:** Establish regular communication with regulatory authorities to stay informed about potential policy changes.
- **Modular Design:** Develop the system with a modular architecture to allow easy updates to specific components without affecting the entire system.
- **Compliance Monitoring:** Assign a team member to track transport and data privacy regulations, ensuring the system remains compliant.

Action Plan:

- Schedule bi-monthly meetings with the Riyadh transport authority to discuss regulatory updates.
- Implement a change management process to quickly adapt to new compliance requirements.
- Conduct quarterly compliance audits to verify adherence to current regulations.

2. Supplier or Vendor Delays

Mitigation Strategy:

- **Vendor Diversification:** Identify and contract with multiple suppliers for critical hardware to reduce dependency on a single vendor.

- **Clear Contracts:** Include strict timelines and penalties for delays in vendor agreements.
- **Buffer Scheduling:** Incorporate contingency time in the project timeline to accommodate potential delays.

Action Plan:

- Shortlist at least two backup vendors for each hardware component by the end of the planning phase.
- Negotiate contracts with clear delivery deadlines and milestones, to be finalized before development starts.
- Add a two-week buffer for hardware integration in the project schedule.

3. Public Resistance to Change

Mitigation Strategy:

- **User Education Campaigns:** Launch awareness campaigns to educate passengers about the benefits and ease of the digital system.
- **Gradual Rollout:** Introduce the system in phases, allowing users to transition slowly while maintaining some traditional options initially.
- **Feedback Mechanism:** Include a help center and feedback feature in the app to address user concerns promptly.

Action Plan:

- Develop a marketing plan for social media and in-station displays to highlight system benefits, starting one month before launch.
- Pilot the digital ticketing system in one metro line for three months before full deployment.
- Integrate a "Help Center" feature in the mobile app with FAQs and live chat support, to be ready at launch.

Risk Monitoring:

1. Continuous Monitoring System

- To keep an eye on all risks that have been identified, use tracking tools like Excel-based risk registers, Microsoft Project, or Primavera Risk Analysis.
- To reflect the risk status (New, Ongoing, Resolved), keep an updated risk log.
- **Important metrics consist of:**
 - ❖ The proportion of significant risks that remain unresolved.
 - ❖ Initial reaction time to novel risks.
 - ❖ Rate of recurrence of risks that have already been identified.

2. Key Risks

- Failure in central signaling/control system.
- Delays from suppliers delivering essential components.
- Cost fluctuation due to material price changes or currency rates.
- Natural events such as sandstorms or flooding.

3. Actions

- Weekly meetings with team leads to review active risks and assign responsibilities.
- Bi-weekly reporting summarizing risk status, severity, and mitigation progress.
- Monthly executive summary submitted to senior management.

4. Escalation Triggers

- Three or more security or safety incidents in a single week.
- Any critical system (e.g., signaling) fails twice in a week.
- More than 5% deviation from planned budget within a month.

5. Response Plans

- **Contingency Plans:** Prepare a quick-response task force to address failures in major systems (signal, power, etc.).
- **Backup:** Daily backup of all the databases is stored offsite.
- **Alternative:** Ensure each critical component has a backup supplier.
- **Preventive measures:** It involves building awareness within employees about cyber and physical threats.

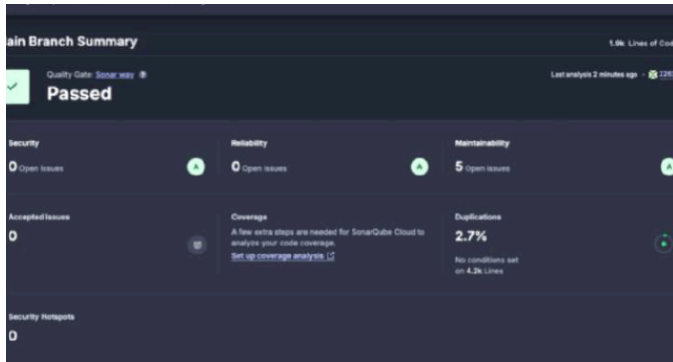
6. Improvement

- Monthly review of risk response effectiveness and policy updates.
- Reassessment of unresolved risks and reclassification if necessary.
- Quarterly workshops for project teams to raise awareness and learn from past incidents.

1- Code Quality Analysis:

1- overall:

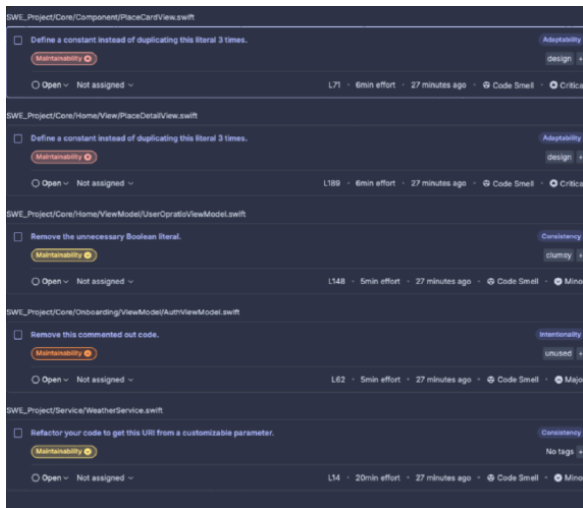
the sonar cloud testing was successful with 5 maintainability issues that will be demonstrated later on, and the 1900 lines of code came with only 2.7% duplication.



Also we got A on security which indicates that the auth solutions were implemented well.

2- maintainability issues:

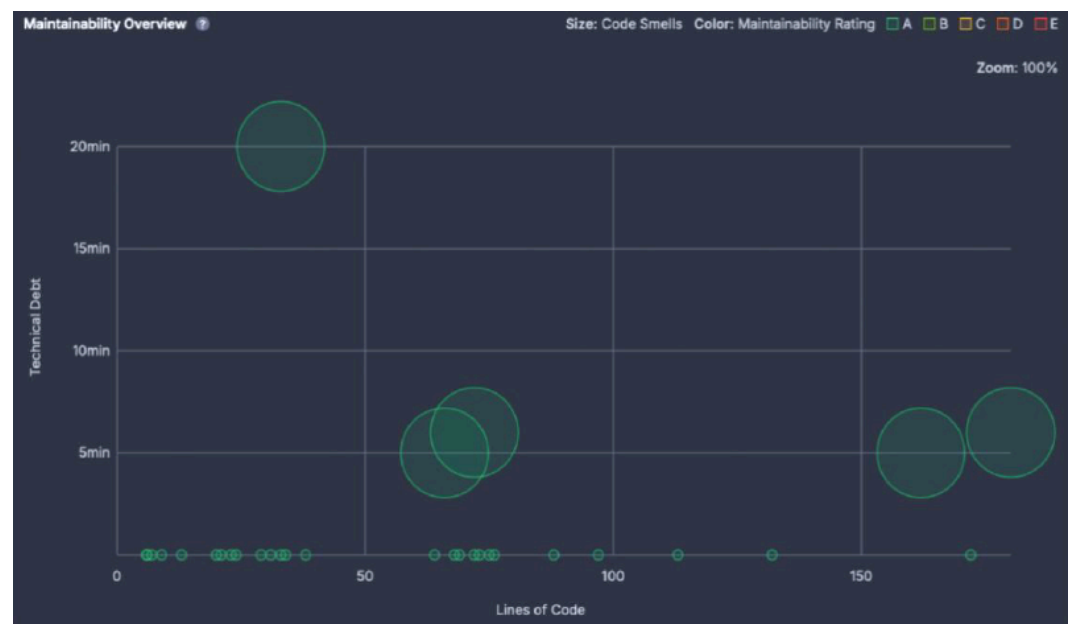
There are only two high maintainability issues which we will attempt to fix them later on.



3- maintainability overview:

The code smells long time risks. the closer the bubble's color to red the higher the risk of technical debt.

bubble size indicates code smell volume, and each bubble position represents the estimated time to address the code smells. the small ones below are the best



2 - ENHANCED SECURITY IMPLEMENTATION:

Description: when user try to signs up, an phone verification is sent. this ensures the email belongs to the user and prevents an unauthorized access

Stage 1: Sign-Up with phone Verification

```
class FirebaseAuthManager extends AuthManager
  with
    EmailSignInManager,
    GoogleSignInManager,
    AppleSignInManager,
    AnonymousSignInManager,
    JwtSignInManager,
    GithubSignInManager,
    PhoneSignInManager {
  // Set when using phone verification (after phone number is provided).
  String? _phoneAuthVerificationCode;
  // Set when using phone sign in in web mode (ignored otherwise).
  ConfirmationResult? _webPhoneAuthConfirmationResult;
  FirebaseAuthManager phoneAuthManager = FirebaseAuthManager();

  @override
  Future signOut() {
    return FirebaseAuth.instance.signOut();
  }

  @override
  Future deleteUser(BuildContext context) async {
    try {
      if (!loggedIn) {
        print('Error: delete user attempted with no logged in user!');
        return;
      }
      await currentUser?.delete();
    } on FirebaseAuthException catch (e) {
      if (e.code == 'requires-recent-login') {
        ScaffoldMessenger.of(context).hideCurrentSnackBar();
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text(
              'Too long since most recent sign in. Sign in again before deleting your account.'), // Text // SnackBar
          );
      }
    }
  }
}
```

Stage 2: auto Verification Check:

Description: A auto function to check if the user has verified their phone, This is called before allowing the user to proceed further in the app.

```
// If you've implemented auto-verification, navigate to home page or
// onboarding page here manually. Uncomment the lines below and replace
// DestinationPage() with the desired widget.
// await Navigator.push(
//   context,
//   MaterialPageRoute(builder: (_) => DestinationPage()),
// );
verificationFailed: (e) {
  phoneAuthManager.update(() {
    phoneAuthManager.triggerOnCodeSent = false;
    phoneAuthManager.phoneAuthError = e;
  });
  completer.complete(false);
},
codeSent: (verificationId, _) {
  phoneAuthManager.update(() {
    phoneAuthManager.phoneAuthVerificationCode = verificationId;
    phoneAuthManager.triggerOnCodeSent = true;
    phoneAuthManager.phoneAuthError = null;
  });
  completer.complete(true);
},
codeAutoRetrievalTimeout: (_) {},
);
return completer.future;
}
```

Stage 3: Sign-Up with Email

Description: When a user signs up, an email verification is sent. This ensures the email belongs to the user and prevents unauthorized access.

```
1  import 'package:firebase_auth/firebase_auth.dart';
2
3  Future<UserCredential?> emailSignInFunc(
4    String email,
5    String password,
6  ) =>
7    FirebaseAuth.instance
8      .signInWithEmailAndPassword(email: email.trim(), password: password);
9
10 Future<UserCredential?> emailCreateAccountFunc(
11   String email,
12   String password,
13 ) =>
14   FirebaseAuth.instance.createUserWithEmailAndPassword(
15     email: email.trim(),
16     password: password,
17   );
18
```


3 - Reliability Measurement:

1. Data:

- Total requests: **200**
- Total failures: **3**
- Total downtime: **59.79 minutes**
- Average restart time: **19.93 minutes**
- Testing period: 4.5 hours = **270 minutes**

2. Probability of Failure on Demand (POFOD):

POFOD = Number of Failures / Total Requests

- Description: Measures the likelihood of a failure occurring when a request is made.
- Calculation: **POFOD** = $3 / 200 \approx 0.015$ (**1.15%**)

3. Rate of Occurrences of Failures (ROCOF):

ROCOF = Number of failures / Testing period (minutes)

- Description: Indicates the failure rate per minute.
- Calculation: **POFOD** = $3 / 270 \approx 0.011111$ **failures per minute**

4. Mean Time Between Failures (MTBF):

MTBF = Testing period (minutes) / Number of Failures

- Description: Indicates the average time between two consecutive failures.
- Calculation: **MTBF** = $270 / 3 = 90$ **minutes**

1. Availability:

Availability = Uptime / All Time

- Description: The percentage of time the system is operational.
- Calculation:

Uptime = All Time - Downtime = 270 - (3 * 19.93) = **210.21 minutes**

Availability = 210.21 / 270 $\approx$ **0.7785 (77.85%)**

4 - Asset Analysis

This section identifies and evaluates critical assets associated with the system. Understanding these assets is essential to determine the potential impact of threats and vulnerabilities.

4.1 Asset Categories

4.1.1 Hardware Assets

- Servers: Host applications and databases; essential for system operations. A compromise could result in significant data loss or service disruption.
- Workstations and Laptops: Used by employees to access company systems. These are susceptible to malware and unauthorized access.
- Network Equipment: Includes routers, switches, and firewalls that ensure proper data flow and network segmentation.
- Mobile Devices: Used for remote access; pose increased risks due to mobility and weaker physical controls.

4.1.2 Software Assets

- Operating Systems (e.g., Windows, Linux): Core platform for system operations; critical for maintaining system functionality and security.
- Business Applications: ERP, CRM, and other productivity software used for business processes.
- Security Software: Antivirus, firewalls, intrusion detection systems (IDS), and endpoint protection software used to safeguard the system.

4.1.3 Data Assets

- Customer Data: Personally identifiable information (PII), financial records, and communication logs; highly sensitive and valuable.
- Corporate Intellectual Property (IP): Source code, designs, and strategic documents; loss could damage competitive positioning.
- Operational Data: System logs, analytics, and procedural documentation used for auditing and decision-making.
- Backup Data: Critical for disaster recovery and business continuity in case of data loss or ransomware attack.

4.1.4 Personnel Assets

- IT Staff: Administer and protect technical infrastructure; high-level access makes them prime targets.
- Executives and Management: Hold sensitive company strategy and financial information.
- General Employees: Often the weakest link; require regular training to prevent social engineering and phishing attacks.

4.2 Asset Valuation and Criticality

Assets are evaluated based on their importance to Confidentiality, Integrity, and Availability (CIA), and assigned a criticality rating.

Asset Type	Example	CIA Priority	Criticality Rating
Data	Customer PII	High Confidentiality	High
Hardware	Application Server	High Availability	High
Software	ERP System	High Integrity & Availability	High
Personnel	IT Administrator	High Confidentiality	High

Extra Protection:

Encrypt Sensitive Data

```
public static byte[] EncryptStringToBytes_Aes(string plainText, byte[] key, byte[] iv)
{
    using (Aes aes = Aes.Create())
    {
        aes.Key = key;
        aes.IV = iv;
        ICryptoTransform encryptor = aes.CreateEncryptor(aes.Key, aes.IV);
        using (MemoryStream ms = new MemoryStream())
        {
            using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
            {
                using (StreamWriter sw = new StreamWriter(cs))
                {
                    sw.Write(plainText);
                }
            }
            return ms.ToArray();
        }
    }
}
```

5 - White-Box Testing:

This is the code from phase 2

```
verificationFailed: (e) {  
    phoneAuthManager.update() {  
        phoneAuthManager.triggerOnCodeSent = false;  
        phoneAuthManager.phoneAuthError = e;  
    }); completer.complete(false); },  
codeSent: (verificationId, _) {  
    phoneAuthManager.update() {  
        phoneAuthManager.phoneAuthVerificationCode = verificationId;  
        phoneAuthManager.triggerOnCodeSent = true;  
        phoneAuthManager.phoneAuthError = null;  
    });  
    completer.complete(true);  
},
```

here the node and the edges

N1: Start

N2: verificationFailed handler

N3: phoneAuthManager.update() with error

N4: completer.complete(false)

N5: codeSent handler

N6: phoneAuthManager.update() with verificationId

N7: completer.complete(true)

N8: End

edges

N1 → N2 (if verification fails)

N2 → N3 → N4 → N8

N1 → N5 (if code is sent)

N5 → N6 → N7 → N8

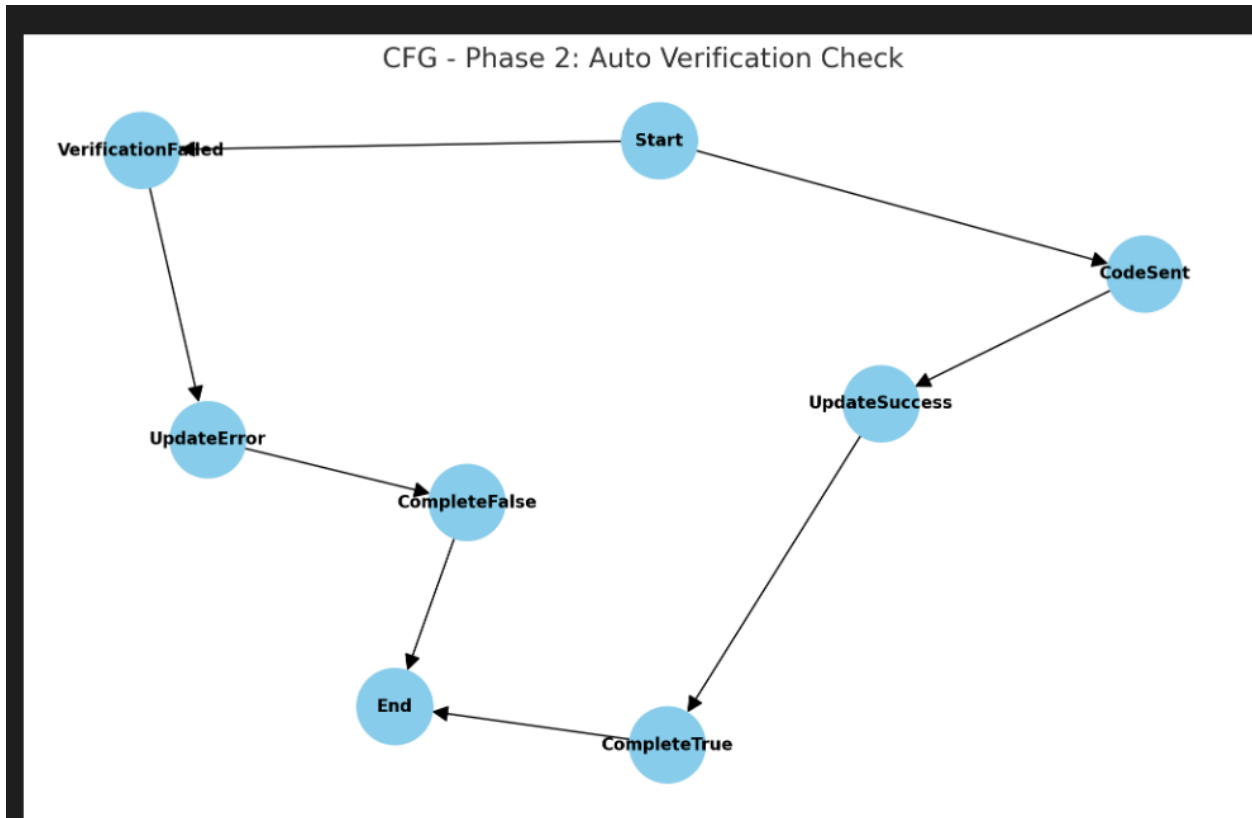
Test Cases for Auto Verification Check:

TC1: Trigger **verificationFailed** with an error.

- Expected: **triggerOnCodeSent** = **false**, **phoneAuthError** = **e**, completes with false.

TC2: Trigger **codeSent** with valid ID.

- Expected: Verification code is set, **triggerOnCodeSent** = **true**, completes with true.



```
Future<UserCredential> emailSignInFunc(String email, String password) {  
    return FirebaseAuth.instance  
        .signInWithEmailAndPassword(email: email.trim(), password: password);  
}  
  
Future<UserCredential> emailCreateAccountFunc(String email, String password) {  
    return FirebaseAuth.instance  
        .createUserWithEmailAndPassword(email: email.trim(), password: password);  
}
```

N1: Start

N2: emailSignInFunc

N3: signInWithEmailAndPassword

N4: emailCreateAccountFunc

N5: createUserWithEmailAndPassword

N6: End

the edges

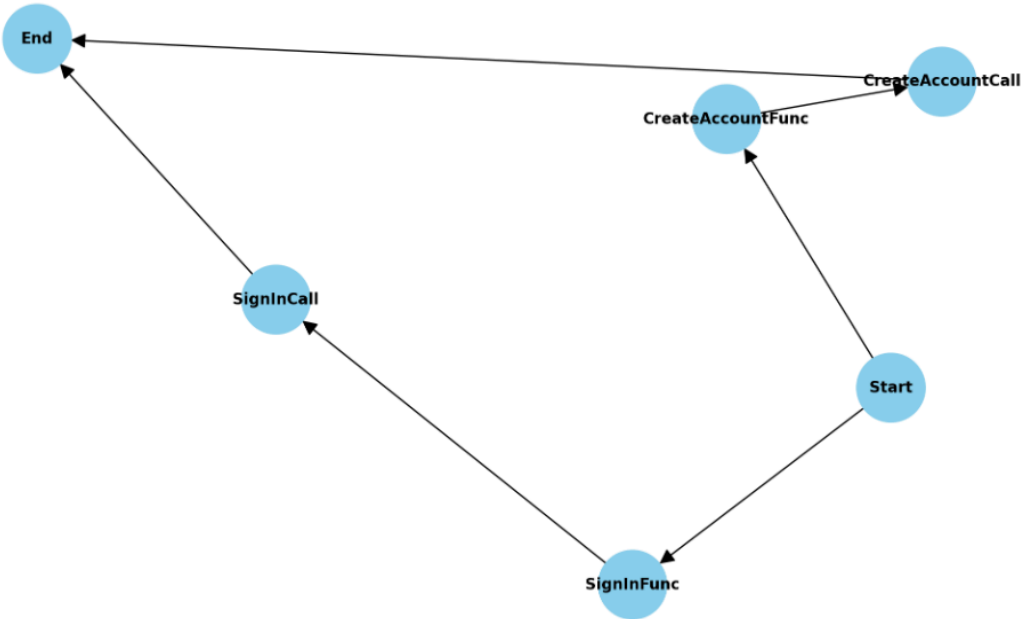
N1 → N2 → N3 → N6 (sign-in path)

N1 → N4 → N5 → N6 (sign-up path)

Test Cases for Email Sign-Up

1. TC3: Call **emailSignInFunc()** with valid credentials.
2. TC4: Call **emailSignInFunc()** with invalid credentials (simulate error).
3. TC5: Call **emailCreateAccountFunc()** with new email.
4. TC6: Call **emailCreateAccountFunc()** with already existing email (simulate error).

CFG - Phase 3: Email Sign-Up



6 - Black-Box Testing:

Test Case ID	Requirement	Input	Expected Output
TC-01	User Registration	Valid name, email, password	Registration successful and account is created
TC-02	User Registration	Email already registered in the system	Registration is blocked, message shown: “Email already exists”
TC-03	Login	Registered email and correct password	User successfully logged in
TC-04	Login	Valid email, wrong password	Login fails, error shown after limited attempts
TC-05	Access Contact Us	Click on Contact Us in the app	support options are displayed
TC-06	Ticket Purchase	Valid credit card, sufficient balance	Ticket is purchased, balance updated, confirmation shown
TC-07	Ticket Purchase	Expired credit card	Transaction rejected, error message displayed
TC-08	Change Profile Info	Update name or phone number in user profile	Changes saved, confirmation shown
TC-09	Logout	Tap logout in settings	User is logged out and redirected to login screen
TC-10	Live Map	Open live map feature	Real-time train station locations and routes are displayed accurately
TC-11	Dark Mode	Enable dark mode in settings	App switches to dark theme
TC-12	Light Mode	Enable light mode in settings	App switches to light theme

6 - AUTOMATED TESTING:

Objective:

Integrating a basic Flutter Test with Github Actions CI/CD to verify stability

Dart test:

We created “app_test.dart” file to verify that the test environment works as intended.

it contains a basic assertion that always passes

```
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('CI/CD Pipeline sanity test', () {
    expect(2 + 2, equals(4));
  });
}
```

CI/CD Configuration:

Github Actions automatically triggers test executions with every change or push to the main branch

the “test.yml” file includes:

- flutter pub get
- flutter test --coverage
- Artifact upload for code coverage

Test results:

Workflow name: riyadh-metro

Execution time: 57 Seconds

Status: Success

Test Passed: 1

Test Failed: 0

Artifact generated: yes